

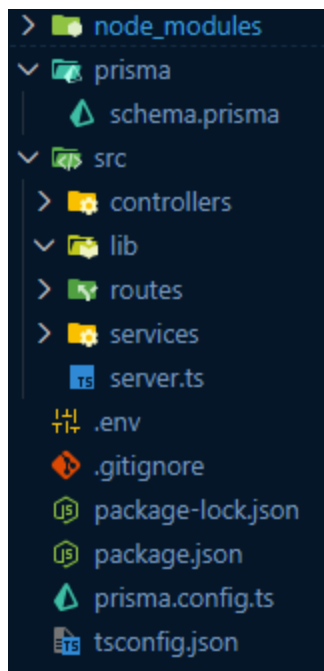
Aplicações e Serviços Web 2025/2026

TP5: Arquitetura backend e separação de camadas

O objetivo desta aula é organizar o código seguindo o padrão de responsabilidades separadas:

- **Server:** Configuração global do servidor e registo de rotas.
- **Routes:** Define os caminhos (endpoints) e delega a execução aos controllers.
- **Controllers:** Gere o protocolo HTTP e valida dados de entrada.
- **Services:** Contém a lógica de negócio e comunica com a base de dados (Prisma).
- **Models:** Definição da estrutura da base de dados no `schema.prisma`.
- **Lib:** Configurações de bibliotecas (e.g., Prisma)

Exemplo de estrutura de pastas:



Nota: Para este guião irá necessitar de ter todas as ferramentas dos guiões anteriores já instaladas.

1. Organização por camadas

1. Vamos começar por criar o conjunto de pastas que iremos necessitar para organizar o código em diferentes camadas e separar as responsabilidades. Para isso deve criar uma pasta `src` na raiz do projeto e de seguida criar as seguintes subpastas: `controllers`, `lib`, `routes`, `services`.
2. De seguida crie um novo ficheiro `prisma.ts` (dentro da pasta `src/lib`) para que tenhamos apenas uma instância única de acesso à base de dados e adicione o seguinte pedaço de código:

```
import { PrismaClient } from '@prisma/client';
import { PrismaPg } from '@prisma/adapters-pg';
import dotenv from 'dotenv';

dotenv.config();

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
});
export const prisma = new PrismaClient({adapter});
```

Nota: O `import` e o `export` são comandos usados para reutilizar código entre ficheiros, nomeadamente funções e variáveis.

3. De seguida vamos adicionar o nosso modelo que representa os utilizadores, acrescentando o seguinte pedaço de código ao `schema.prisma`:

```
model User {
  id      Int      @id @default(autoincrement())
  fullName String
  email   String   @unique
  createdAt DateTime @default(now())
}
```

4. Vamos criar o ficheiro `userService.ts` que irá tratar da comunicação com a base de dados para o modelo `User`:

```
import { prisma } from '../lib/prisma.js';

export const findAllUsers = async () => {
  return await prisma.user.findMany();
}
```

```
};

export const createUser = async (fullName: string, email: string) => {
  return await prisma.user.create({
    data: { fullName, email }
  });
};
```

5. Vamos criar o ficheiro UserController.ts que irá tratar da gestão dos pedidos e respostas para a rota users:

```
import { type Request, type Response } from 'express';
import * as userService from '../services/userService.js';

export const getUsers = async (req: Request, res: Response) => {
  const users = await userService.findAllUsers();
  res.json(users);
};

export const createUserController = async (req: Request, res: Response) => {
  try {
    const { fullName, email } = req.body;
    const newUser = await userService.createUser(fullName, email);
    res.status(201).json(newUser);
  } catch (error) {
    res.status(400).json({ error: "Invalid data or email already exists" });
  }
};
```

6. Vamos criar o ficheiro UserRoutes.ts que irá incluir os endpoints associados ao recursos users:

```
import { Router } from 'express';
import { getUsers, createUserController } from '../controllers/userController.js';

const router = Router();
router.get('/', getUsers);
router.post('/', createUserController);

export default router;
```

7. Vamos agora criar o server.ts que será o ponto de acesso à nossa API:

```
import express from 'express';
import userRoutes from './routes/userRoutes.js';

const app = express();
app.use(express.json());

app.use('/api/users', userRoutes);

app.listen(3000, () => console.log("Server running on http://localhost:3000"));
```

8. Execute o seguinte comando no terminal para acrescentar o modelo criado na sua base dados:

npx prisma migrate dev --name criacao_utilizadores

9. Execute o seguinte comando no terminal para atualizar o módulo Prisma Client:

npx prisma generate

Nota: Deverá executar o comando sempre que atualizar o ficheiro schema.prisma para que as alterações realizadas sejam refletidas no Prisma Client.

10. No terminal, execute o script para ligar o servidor:

npx tsx watch src/server.ts

Nota: O watch permite que o servidor reinicialize sozinho sempre que guardar o ficheiro. Caso ainda não tenha instalado o tsx (executor typescript para node) , deverá primeiro executar o seguinte comando npm install -D tsx .

11. Realizar vários pedidos CRUD à api criada e de seguida verifique se alterações foram devidamente propagadas na base de dados. Verifique também diretamente no PostgreSQL.

2. Relação 1 para muitos com modelos do Prisma

1. Vamos adicionar ao exercício anterior mais duas entidades (Author e Book) de forma a suportar relações mais complexas nos nossos dados.
2. De seguida vamos adicionar ao schema.prisma dois novos modelos:

```
model Author {
  id Int @id @default(autoincrement())
  name String
  books Book[] // Relation 1:N
}
```

```

model Book {
  id    Int    @id @default(autoincrement())
  title String
  author Author @relation(fields: [authorId], references: [id])
  authorId Int
}

```

3. Vamos criar o ficheiro bookService.ts que irá tratar da comunicação com a base de dados para o modelo Book:

```

import { prisma } from '../lib/prisma.js';

export const findAllBooks = async () => {
  return await prisma.book.findMany({
    include: {
      author: true
    }
  });
};

export const createBook = async (title: string, authorId: number) => {
  return await prisma.book.create({
    data: {
      title,
      authorId
    }
  });
};

```

4. Vamos criar o ficheiro bookController.ts que irá tratar da gestão dos pedidos e respostas para a rota books:

```

import { Request, Response } from 'express';
import * as bookService from '../services/bookService.js';

export const getBooks = async (req: Request, res: Response) => {
  try {
    const books = await bookService.findAllBooks();
    res.json(books);
  } catch (error) {
    res.status(500).json({ error: "Error fetching books" });
  }
};

```

```
export const createBookController = async (req: Request, res: Response) => {
  const { title, authorId } = req.body;
  try {
    const newBook = await bookService.createBook(title, authorId);
    res.status(201).json(newBook);
  } catch (error) {
    res.status(400).json({ error: "Invalid data or author not found" });
  }
};
```

5. Vamos criar o ficheiro bookRoutes.ts que irá incluir os endpoints associados ao recursos books:

```
import { Router } from 'express';
import { getBooks, createBookController } from '../controllers/bookController.js';

const router = Router();

router.get('/', getBooks);
router.post('/', createBookController);

export default router;
```

6. Vamos agora adicionar ao server.ts o seguinte pedaço de código:

```
import bookRoutes from './routes/bookRoutes.js';

app.use('/api/books', bookRoutes);
```

7. Execute o seguinte comando no terminal para acrescentar o modelo criado na sua base dados:

npx prisma migrate dev --name criacao_books_authors

8. Execute o seguinte comando no terminal para atualizar o módulo Prisma Client:

npx prisma generate

Nota: Deverá executar o comando sempre que atualizar o ficheiro schema.prisma para que as alterações realizadas sejam refletidas no Prisma Client.

9. No terminal, execute o script para ligar o servidor:

npx tsx watch src/server.ts

10. Realizar vários pedidos CRUD à api criada e de seguida verifique se alterações foram devidamente propagadas na base de dados. Verifique também diretamente no PostgreSQL.

3. Exercícios

Adicione ao exemplo anterior os ficheiros e pedaços de código adequados para a resolução dos exercícios seguintes.

Nota: Todos os exercícios devem ser implementados respeitando a separação de camadas: Routes, Controllers e Services.

Exercício 1

Crie a estrutura necessária para gerir o recurso author. Deverá suportar uma pesquisa por todos os autores e adicionar um novo autor.

Exercício 2

Adicione a estrutura necessária para suportar uma funcionalidade de pesquisa para o recurso Book que permita procurar livros cujo título contenha uma palavra-chave. Deverá ser possível utilizar esta pesquisa através da rota `books/search`.

Exercício 3

Adicione a estrutura necessária para que seja possível pesquisar um autor pelo seu id. A lista de livros associados deverá fazer parte dos campos a retornar.

Nota: poderá ser útil o comando `include: { books: true }`

Exercício 4

Ao criar um novo livro deverá verificar se o id do autor existe. Se não existir, lance um erro. O Controller deve capturar o erro e responder com 404 Not Found.

Exercício 5

Adicione a estrutura necessária para que seja possível obter os 3 livros mais recentes (baseado no ID descendente).

5. Tutoriais

Deve realizar os tutoriais de suporte abaixo e consultar a documentação oficial:

- Prisma
 - [Relação 1 para N](#)
- Node
 - [tsx](#)